Department of Computer Science and Engineering Assignment Questions

Course Code & Name: CSA0927 – Programming in Java

Assignment Title:	Government Public Grievance Management System using Java
Course Outcome(s) – CO:	**CO1:** Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) **CO2:** Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) **CO3:** Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 16 – Peace, Justice and Strong Institutions SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities SDG 11 – Sustainable Cities and Communities

1.Problem Statement

A district government office receives numerous public complaints related to roads, water supply, sanitation, streetlights, public transport, and other civic services. A manual process makes it difficult to register complaints, assign them to the appropriate departments, track their status, maintain complaint history, and identify pending cases. The proposed system automates these activities through a menu-driven Java application.

The system maintains citizen details, complaint records, department details, responsible officers, priority levels, statuses, and history. Different user roles are represented through inheritance and method overriding. The application uses Java collections to manage records and uses synchronized access with multiple threads to process simultaneous complaint submissions safely. Custom exceptions manage invalid inputs, duplicate complaints, unavailable departments, incorrect status transitions, and missing records.

2. Objective

The main objective is to design and implement a reliable Java-based Government Public Grievance Management System that improves the registration, allocation, tracking, and reporting of public complaints.

Objective	**Expected outcome**
Citizen management	Register citizens and maintain unique citizen identifiers.
Complaint management	Submit complaints with department, description, priority, status, and history.
Department allocation	Assign complaints only to officers belonging to the relevant department.
Tracking and reporting	Track individual complaints and generate department-wise, status-wise, and pending reports.
Java learning outcomes	Demonstrate OOP, collections, generics, iterators, threads, synchronization, communication, and exceptions.

3. Requirements and Environment Used

Category	Requirement / choice
Hardware	Any computer capable of running a Java Development Kit and command-line application.
Operating system	Windows, Linux, or macOS.
Programming language	Java 21; the code uses standard Java SE libraries only.
Development tools	JDK, command prompt or terminal, and any Java IDE such as IntelliJ IDEA, Eclipse, or NetBeans.
Input method	Keyboard input through java.util.Scanner.
Storage model	In-memory storage using HashMap, HashSet, ArrayList, Map, Set, List, and Iterator.
Security and validation	Encapsulated fields, input validation, role-oriented classes, and custom checked exceptions.
Learning outcomes	CO1: OOP; CO2: collections and generics; CO3: multithreading, synchronization, communication, and exceptions.

This submission is a Java-only implementation. The executable application, domain model, collections, user interface, validation, exception handling, reports, multithreading, synchronization, and inter-thread communication are all written in Java using Java SE classes. No Python, JavaScript, HTML, CSS, database query language, or external framework is required to compile or execute the application.

The assignment brief maps the system to SDG 16 (Peace, Justice and Strong Institutions), SDG 9 (Industry, Innovation and Infrastructure), SDG 10 (Reduced Inequalities), and SDG 11 (Sustainable Cities and Communities).

4. Design / Proposed Solution

The proposed solution is a Java SE console application organized around Java classes, enumerations, collections, and a service layer. User is an abstract Java superclass. Citizen, Officer, Supervisor, and Administrator inherit from User and override getRole(), demonstrating runtime polymorphism.

Complaint encapsulates its data and controls assignment and valid forward-only status changes. ManagementService owns the Java collections and provides synchronized operations.

Java implementation mapping: GrievanceManagementSystem is the public class containing main() and the Scanner-driven menu; User, Citizen, Officer, Supervisor, and Administrator form the inheritance hierarchy; Department and Complaint are domain classes; Priority and Status are Java enums; ManagementService is the synchronized business-service class; and the custom exception classes extend java.lang.Exception.

Class / component	**Responsibility**	**Java concept demonstrated**
User	Common identity and contact information for system users.	Abstraction and encapsulation
Citizen	Stores citizen email and role information.	Inheritance and overriding
Officer	Stores department-specific officer information.	Inheritance and composition
Supervisor / Administrator	Represent higher-level user roles.	Polymorphism
Department	Stores department name and officer IDs.	HashSet and encapsulation
Complaint	Stores complaint details, priority, status, officer, and history.	Encapsulation, ArrayList, enum
ManagementService	Registers users, submits complaints, assigns officers, updates status, and creates reports.	HashMap, synchronization, generics
GrievanceManagementSystem	Provides the menu-driven user interface.	Scanner, control structures, exception handling

5. Algorithm, Pseudocode and Flowchart

5.1 Complaint Submission Algorithm

Algorithm: SUBMIT_COMPLAINT

Input: citizenId, department, description, priority
Output: Newly created complaint

1. START
2. Acquire the shared synchronization lock.
3. Check whether the given citizenId exists.
4. If the citizen does not exist, raise InvalidInputException.
5. Normalize the department name to uppercase.
6. Check whether the department exists.
7. If the department is unavailable, raise DepartmentUnavailableException.
8. Check whether the complaint description is empty.
9. If the description is empty, raise InvalidInputException.
10. Validate the complaint priority.
11. Search the existing complaints.
12. Check whether an active complaint with the same citizen, department and description already exists.
13. If a duplicate is found, raise DuplicateComplaintException.
14. Generate the next complaint ID using AtomicInteger.
15. Create a new Complaint object.
16. Set the initial complaint status to SUBMITTED.
17. Store the complaint in the HashMap.

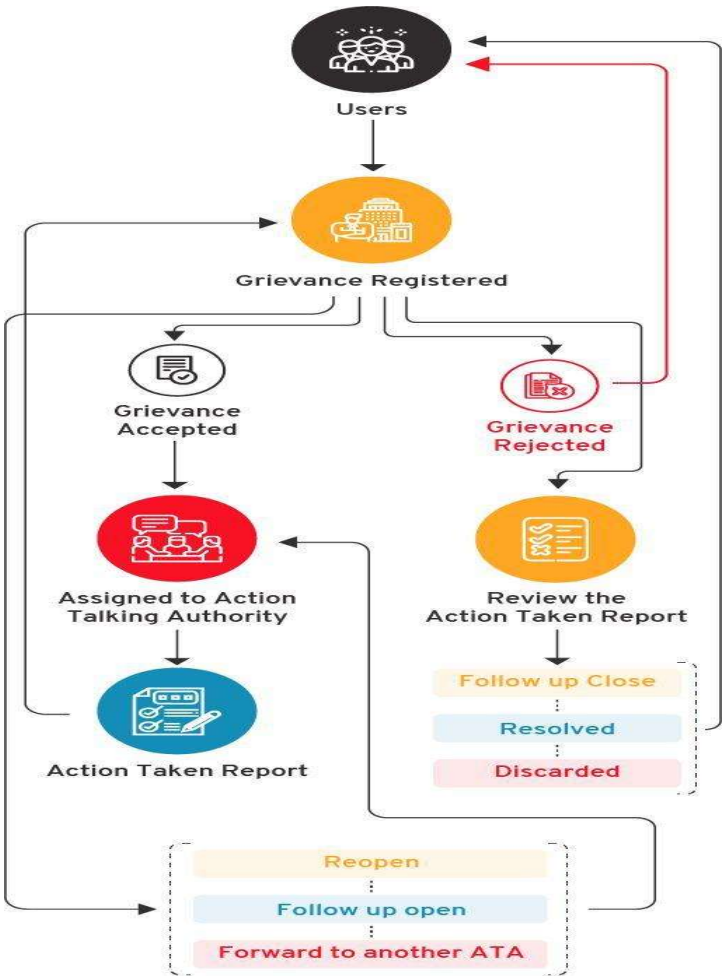
18. Notify waiting threads using notifyAll().

19. Release the shared lock.

20. Return the newly created complaint.

21. STOP

5.2 Text Flowchart



6. Implementation / Source Code

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
public class Main {
    private final Scanner scanner = new Scanner(System.in);
    private final ManagementService service = new ManagementService();
    public static void main(String[] args) {
        Main app = new Main();
        app.service.seedDepartments();
        app.run();
    }
    private void run() {

        boolean exit = false;

        while (!exit) {
            printMainMenu();
            int choice = readInt("Enter choice: ");
            try {
                switch (choice) {
                    case 1:
                        registerCitizen();
                        break;
                    case 2:
                        submitComplaint();
```

```
            break;
        case 3:
            assignOfficer();
            break;
        case 4:
            updateStatus();
            break;
        case 5:
            trackComplaint();
            break;
        case 6:
            service.printAllComplaints();
            break;
        case 7:
            service.printReports();
            break;
        case 8:
            service.simulateConcurrentSubmissions();
            break;
        case 9:
            exit = true;
            break;
        default:
            System.out.println(
                    "Invalid menu choice."
            );
    }
```

```
        } catch (GrievanceException e) {

            System.out.println(
                    "Operation failed: " + e.getMessage()
            );

        } catch (Exception e) {

            System.out.println(
                    "Unexpected error: " + e.getMessage()
            );
        }
    }

    System.out.println();
    System.out.println(
            "Thank you for using the Government Public Grievance Management System."
    );
}
private void printMainMenu() {

    System.out.println();
    System.out.println(
            "============================================================="
    );
    System.out.println(
```

```
            "       GOVERNMENT PUBLIC GRIEVANCE MANAGEMENT SYSTEM"
    );
    System.out.println(
            "============================================================="
    );
    System.out.println("1. Register citizen");
    System.out.println("2. Submit complaint");
    System.out.println("3. Assign officer");
    System.out.println("4. Update complaint status");
    System.out.println("5. Track complaint");
    System.out.println("6. Display all complaints");
    System.out.println("7. Generate reports");
    System.out.println("8. Simulate concurrent submissions");
    System.out.println("9. Exit");
    System.out.println(
            "============================================================="
    );
}
private void registerCitizen()
        throws GrievanceException {
    System.out.println();
    System.out.println(
            "---------------- REGISTER CITIZEN ----------------"
    );
    String name =
            readText("Citizen name: ");
    String phone =
```

```
            readText("Phone number: ");
        String email =
            readText("Email: ");
        service.registerCitizen(
            name,
            phone,
            email
        );
        System.out.println(
            "Citizen registered successfully."
        );
    }
    private void submitComplaint()
            throws GrievanceException {

        System.out.println();
        System.out.println(
            "---------------- SUBMIT COMPLAINT ----------------"
        );

        String citizenId =
            readText("Citizen ID: ");

        String department =
            readText(
                "Department "
                + "(Roads/Water/Sanitation/"
```

```
            + "Streetlights/Transport): "
        );

String description =
        readText(
            "Complaint description: "
        );

String priority =
        readText(
            "Priority "
            + "(LOW/MEDIUM/HIGH/URGENT): "
        );

Priority selectedPriority;
try {
    selectedPriority =
            Priority.valueOf(
                priority
                        .trim()
                        .toUpperCase(
                            Locale.ROOT
                        )
            );
} catch (IllegalArgumentException e) {

    throw new InvalidInputException(
```

```
                "Invalid priority. Please use "
                + "LOW, MEDIUM, HIGH or URGENT."
            );
        }

        Complaint complaint =
                service.submitComplaint(
                        citizenId,
                        department,
                        description,
                        selectedPriority
                );

        System.out.println();
        System.out.println(
                "Complaint submitted successfully."
        );

        System.out.println(
                "Generated Complaint ID: "
                        + complaint.getComplaintId()
        );
    }
    private void assignOfficer()
            throws GrievanceException {

        System.out.println();
```

```
    System.out.println(
            "---------------- ASSIGN OFFICER ----------------"
    );
    int complaintId =
            readInt("Complaint ID: ");

    String officerId =
            readText("Officer ID: ");

    service.assignOfficer(
            complaintId,
            officerId
    );

    System.out.println(
            "Officer assigned successfully."
    );
}
private void updateStatus()
        throws GrievanceException {

    System.out.println();
    System.out.println(
            "---------------- UPDATE STATUS ----------------"
    );

    int complaintId =
```

```
        readInt("Complaint ID: ");

String status =
        readText(
                "Status "
                + "(SUBMITTED/ASSIGNED/"
                + "IN_PROGRESS/RESOLVED/CLOSED): "
        );

Status newStatus;

try {

    newStatus =
            Status.valueOf(
                    status
                            .trim()
                            .toUpperCase(
                                    Locale.ROOT
                            )
            );

} catch (IllegalArgumentException e) {

    throw new InvalidStatusException(
            "Invalid status. Please use "
            + "SUBMITTED, ASSIGNED, IN_PROGRESS, "
```

```
                + "RESOLVED or CLOSED."
        );
    }

    service.updateStatus(
            complaintId,
            newStatus
    );

    System.out.println(
            "Complaint status updated successfully."
    );
}

// ============================================================
// TRACK COMPLAINT
// ============================================================

private void trackComplaint()
        throws GrievanceException {

    System.out.println();
    System.out.println(
            "---------------- TRACK COMPLAINT ----------------"
    );

    int complaintId =
```

```
        readInt("Complaint ID: ");

Complaint complaint =
        service.findComplaint(
                complaintId
        );

System.out.println();
System.out.println(
        "================= COMPLAINT DETAILS ================="
);

System.out.println(
        "Complaint ID : "
                + complaint.getComplaintId()
);

System.out.println(
        "Citizen ID    : "
                + complaint.getCitizenId()
);

System.out.println(
        "Department    : "
                + complaint.getDepartmentName()
);
```

```
System.out.println(
        "Description   : "
                + complaint.getDescription()
);

System.out.println(
        "Priority      : "
                + complaint.getPriority()
);

System.out.println(
        "Status        : "
                + complaint.getStatus()
);

System.out.println(
        "Officer ID    : "
                + complaint.getOfficerId()
);

System.out.println(
        "Created At    : "
                + complaint.getCreatedAt()
);

System.out.println(
        "=================================================="
```

```
    );

    System.out.println();
    System.out.println(
            "---------------- COMPLAINT HISTORY ----------------"
    );

    for (String history :
            complaint.getHistory()) {

        System.out.println(history);
    }

    System.out.println(
            "-------------------------------------------------"
    );
}

// ================================================================
// READ INTEGER
// ================================================================

private int readInt(String prompt) {

    while (true) {

        try {
```

```
            System.out.print(prompt);

            int value =
                    scanner.nextInt();

            scanner.nextLine();

            return value;

        } catch (InputMismatchException e) {

            scanner.nextLine();

            System.out.println(
                    "Please enter a valid number."
            );
        }
    }
}

// ================================================================
// READ TEXT
// ================================================================

private String readText(String prompt) {
```

```
        System.out.print(prompt);

        return scanner.nextLine().trim();
    }
}

// ========================================================================
// ABSTRACT USER CLASS
// ========================================================================

abstract class User {

    private final String id;
    private String name;
    private String phone;

    protected User(
            String id,
            String name,
            String phone) {

        this.id =
                requireText(id, "ID");

        this.name =
                requireText(name, "Name");
```

```
    this.phone =
            requireText(phone, "Phone");
}

public String getId() {
    return id;
}

public String getName() {
    return name;
}

public String getPhone() {
    return phone;
}

public void setName(String name) {

    this.name =
            requireText(name, "Name");
}

public void setPhone(String phone) {

    this.phone =
            requireText(phone, "Phone");
}
```

```
public abstract String getRole();


@Override
public String toString() {


    return id
            + " - "
            + name
            + " ("
            + getRole()
            + ")";
}


protected static String requireText(
        String value,
        String field) {


    if (value == null ||
            value.trim().isEmpty()) {


        throw new IllegalArgumentException(
                field + " cannot be empty."
        );
    }


    return value.trim();
```

```
    }
}
class Citizen extends User {

    private final String email;

    public Citizen(
            String id,
            String name,
            String phone,
            String email) {

        super(id, name, phone);

        if (email == null ||
                !email.contains("@") ||
                email.startsWith("@") ||
                email.endsWith("@")) {

            throw new IllegalArgumentException(
                    "Invalid email address."
            );
        }

        this.email =
                email.trim();
    }
```

```
    public String getEmail() {

        return email;
    }

    @Override
    public String getRole() {

        return "Citizen";
    }
}

class Officer extends User {

    private final String departmentName;

    public Officer(
            String id,
            String name,
            String phone,
            String departmentName) {

        super(id, name, phone);

        this.departmentName =
                requireText(
```

```
                departmentName,
                "Department"
            ).toUpperCase(
                Locale.ROOT
            );
    }

    public String getDepartmentName() {

        return departmentName;
    }

    @Override
    public String getRole() {

        return "Officer";
    }
}

class Supervisor extends User {

    public Supervisor(
            String id,
            String name,
            String phone) {

        super(id, name, phone);
```

```
    }

    @Override
    public String getRole() {

        return "Supervisor";
    }
}

class Administrator extends User {

    public Administrator(
            String id,
            String name,
            String phone) {

        super(id, name, phone);
    }

    @Override
    public String getRole() {

        return "Administrator";
    }
}
enum Priority {
```

```
    LOW,
    MEDIUM,
    HIGH,
    URGENT
}

enum Status {

    SUBMITTED,
    ASSIGNED,
    IN_PROGRESS,
    RESOLVED,
    CLOSED
}
class Department {

    private final String name;

    private final Set<String> officerIds =
            new HashSet<>();

    public Department(String name) {

        this.name = name;
    }

    public String getName() {
```

```
    return name;
}

public Set<String> getOfficerIds() {

    return Collections.unmodifiableSet(
            officerIds
    );
}

public void addOfficer(
        String officerId) {

    officerIds.add(
            officerId
    );
}

@Override
public String toString() {

    return "Department: "
            + name
            + ", Officers: "
            + officerIds;
}
```

```
}

class Complaint {

    private final int complaintId;

    private final String citizenId;

    private final String departmentName;

    private final String description;

    private final Priority priority;

    private Status status;

    private String officerId;

    private final List<String> history =
            new ArrayList<>();

    private final LocalDateTime createdAt;

    public Complaint(
            int complaintId,
            String citizenId,
            String departmentName,
```

```
    String description,
    Priority priority) {

this.complaintId =
        complaintId;

this.citizenId =
        citizenId;

this.departmentName =
        departmentName;

this.description =
        description;

this.priority =
        priority;

this.status =
        Status.SUBMITTED;

this.createdAt =
        LocalDateTime.now();

addHistory(
        "Complaint submitted "
        + "with status SUBMITTED"
```

```
    );
}

public int getComplaintId() {

    return complaintId;
}

public String getCitizenId() {

    return citizenId;
}

public String getDepartmentName() {

    return departmentName;
}

public String getDescription() {

    return description;
}

public Priority getPriority() {

    return priority;
}
```

```
public Status getStatus() {

    return status;
}

public String getOfficerId() {

    return officerId;
}

public LocalDateTime getCreatedAt() {

    return createdAt;
}

public List<String> getHistory() {

    return Collections.unmodifiableList(
            history
    );
}

public void assignOfficer(
        String officerId) {

    this.officerId =
```

```
            officerId;

    this.status =
            Status.ASSIGNED;

    addHistory(
            "Officer "
            + officerId
            + " assigned"
    );
}

public void updateStatus(
        Status newStatus)
        throws InvalidStatusException {

    if (newStatus == null) {

        throw new InvalidStatusException(
                "Status cannot be null."
        );
    }

    if (newStatus.ordinal()
            < status.ordinal()) {

        throw new InvalidStatusException(
```

```
                "Status cannot move backwards "
                + "from "
                + status
                + " to "
                + newStatus
                + "."
        );
    }

    this.status =
            newStatus;

    addHistory(
            "Status changed to "
            + newStatus
    );
}

private void addHistory(
        String event) {

    String timestamp =
            LocalDateTime.now().format(
                    DateTimeFormatter.ofPattern(
                            "yyyy-MM-dd HH:mm:ss"
                    )
            );
```

```
        history.add(
                timestamp
                + " - "
                + event
        );
    }

    @Override
    public String toString() {

        return "Complaint{" +
                "id=" + complaintId +
                ", citizen='" + citizenId + '\'' +
                ", department='" + departmentName + '\'' +
                ", description='" + description + '\'' +
                ", priority=" + priority +
                ", status=" + status +
                ", officer='" + officerId + '\'' +
                ", createdAt=" + createdAt +
                '}';
    }
}

class ManagementService {

    private final Map<String, Citizen> citizens =
```

```
        new HashMap<>();

private final Map<String, Department> departments =
        new HashMap<>();

private final Map<String, Officer> officers =
        new HashMap<>();

private final Map<Integer, Complaint> complaints =
        new HashMap<>();

private final AtomicInteger complaintSequence =
        new AtomicInteger(1000);

private final Object lock =
        new Object();

public void seedDepartments() {

    departments.putIfAbsent(
            "ROADS",
            new Department("ROADS")
    );

    departments.putIfAbsent(
            "WATER",
            new Department("WATER")
```

```
);

departments.putIfAbsent(
        "SANITATION",
        new Department("SANITATION")
);

departments.putIfAbsent(
        "STREETLIGHTS",
        new Department("STREETLIGHTS")
);

departments.putIfAbsent(
        "TRANSPORT",
        new Department("TRANSPORT")
);

Officer roadsOfficer =
        new Officer(
                "O101",
                "Anita Rao",
                "9000000001",
                "ROADS"
        );

Officer waterOfficer =
        new Officer(
```

```
            "O102",
            "Bharat Kumar",
            "9000000002",
            "WATER"
        );

officers.put(
        roadsOfficer.getId(),
        roadsOfficer
);

officers.put(
        waterOfficer.getId(),
        waterOfficer
);

departments
        .get("ROADS")
        .addOfficer(
            roadsOfficer.getId()
        );

departments
        .get("WATER")
        .addOfficer(
            waterOfficer.getId()
        );
```

```
}

public void registerCitizen(
        String name,
        String phone,
        String email)
        throws GrievanceException {

    synchronized (lock) {

        if (email == null ||
                email.trim().isEmpty()) {

            throw new InvalidInputException(
                    "Email cannot be empty."
            );
        }

        for (Citizen citizen :
                citizens.values()) {

            if (citizen.getEmail()
                    .equalsIgnoreCase(
                            email.trim()
                    )) {

                throw new DuplicateComplaintException(
```

```
                "A citizen with this "
                + "email already exists."
        );
    }
}

String id =
        "C"
        + (citizens.size() + 1);

try {

    Citizen citizen =
            new Citizen(
                    id,
                    name,
                    phone,
                    email
            );

    citizens.put(
            id,
            citizen
    );

} catch (IllegalArgumentException e) {
```

```
            throw new InvalidInputException(
                    e.getMessage()
            );
        }

        System.out.println(
                "Generated Citizen ID: "
                + id
        );
    }
}

// ================================================================
// SUBMIT COMPLAINT
// ================================================================

public Complaint submitComplaint(
        String citizenId,
        String departmentName,
        String description,
        Priority priority)
        throws GrievanceException {

    synchronized (lock) {

        if (citizenId == null ||
                citizenId.trim().isEmpty()) {
```

```
    throw new InvalidInputException(
            "Citizen ID cannot be empty."
    );
}

citizenId =
        citizenId.trim();

if (!citizens.containsKey(
        citizenId)) {

    throw new InvalidInputException(
            "Citizen ID not found."
    );
}

if (departmentName == null ||
        departmentName.trim().isEmpty()) {

    throw new InvalidInputException(
            "Department cannot be empty."
    );
}

String dept =
        departmentName
```

```
            .trim()
            .toUpperCase(
                Locale.ROOT
            );

if (!departments.containsKey(
        dept)) {

    throw new DepartmentUnavailableException(
            "Department is unavailable."
    );
}

if (description == null ||
        description.trim().isEmpty()) {

    throw new InvalidInputException(
            "Description cannot be empty."
    );
}

if (priority == null) {

    throw new InvalidInputException(
            "Priority cannot be null."
    );
}
```

```
String cleanDescription =
        description.trim();

// ========================================================
// DUPLICATE CHECK
// ========================================================

for (Complaint c :
        complaints.values()) {

    if (c.getCitizenId()
            .equals(citizenId)
            &&
        c.getDepartmentName()
            .equals(dept)
            &&
        c.getStatus()
            != Status.CLOSED
            &&
        c.getStatus()
            != Status.RESOLVED
            &&
        c.getDescription()
            .equalsIgnoreCase(
                    cleanDescription
            )) {
```

```
            throw new DuplicateComplaintException(
                    "A similar active "
                    + "complaint already exists."
            );
        }
    }

    // ========================================================
    // GENERATE COMPLAINT ID
    // ========================================================

    int id =
            complaintSequence
                    .incrementAndGet();

    Complaint complaint =
            new Complaint(
                    id,
                    citizenId,
                    dept,
                    cleanDescription,
                    priority
            );

    complaints.put(
            id,
```

```
            complaint
        );

        // Notify waiting thread
        lock.notifyAll();

        return complaint;
    }
}

// ================================================================
// ASSIGN OFFICER
// ================================================================

public void assignOfficer(
        int complaintId,
        String officerId)
        throws GrievanceException {

    synchronized (lock) {

        Complaint complaint =
                findComplaint(
                        complaintId
                );

        if (officerId == null ||
```

```
        officerId.trim().isEmpty()) {

    throw new InvalidInputException(
            "Officer ID cannot be empty."
    );
}

officerId =
        officerId.trim();

Officer officer =
        officers.get(
                officerId
        );

if (officer == null) {

    throw new InvalidInputException(
            "Officer ID not found."
    );
}

if (!officer
        .getDepartmentName()
        .equals(
                complaint
                        .getDepartmentName()
```

```
            )) {

            throw new InvalidInputException(
                "Officer does not belong "
                + "to the complaint department."
            );
        }

        complaint.assignOfficer(
                officerId
        );
    }
}

// ================================================================
// UPDATE STATUS
// ================================================================

public void updateStatus(
        int complaintId,
        Status status)
        throws GrievanceException {

    synchronized (lock) {

        Complaint complaint =
                findComplaint(
```

```
                complaintId
            );

        complaint.updateStatus(
            status
        );
    }
}

// ================================================================
// FIND COMPLAINT
// ================================================================

public Complaint findComplaint(
        int id)
        throws ComplaintNotFoundException {

    Complaint complaint =
            complaints.get(id);

    if (complaint == null) {

        throw new ComplaintNotFoundException(
            "Complaint ID not found."
        );
    }
```

```
    return complaint;
}

// ============================================================
// DISPLAY ALL COMPLAINTS
// ============================================================

public void printAllComplaints() {

    synchronized (lock) {

        System.out.println();

        System.out.println(
                "================ ALL COMPLAINTS ================"
        );

        if (complaints.isEmpty()) {

            System.out.println(
                    "No complaints available."
            );

            System.out.println(
                    "================================================"
            );
```

```
            return;
        }

        Iterator<Complaint> iterator =
                complaints.values()
                        .iterator();

        while (iterator.hasNext()) {

            System.out.println(
                    iterator.next()
            );

            System.out.println(
                    "------------------------------------------------"
            );
        }

        System.out.println(
                "============================================================"
        );
    }
}

// ================================================================
// GENERATE REPORTS
// ================================================================
```

```
public void printReports() {

    synchronized (lock) {

        Map<String, Integer>
                departmentCounts =
                new HashMap<>();

        Map<Status, Integer>
                statusCounts =
                new EnumMap<>(
                        Status.class
                );

        int pending = 0;

        // ====================================================
        // COUNT COMPLAINTS
        // ====================================================

        for (Complaint c :
                complaints.values()) {

            departmentCounts.put(
                    c.getDepartmentName(),
```

```
            departmentCounts.getOrDefault(
                    c.getDepartmentName(),
                    0
            ) + 1
    );

    statusCounts.put(
            c.getStatus(),

            statusCounts.getOrDefault(
                    c.getStatus(),
                    0
            ) + 1
    );

    if (c.getStatus()
            != Status.RESOLVED
            &&
        c.getStatus()
            != Status.CLOSED) {

        pending++;
    }
}

// ========================================================
// DEPARTMENT REPORT
```

```
// ==================================================

System.out.println();

System.out.println(
        "----------- DEPARTMENT-WISE REPORT -----------"
);

if (departmentCounts.isEmpty()) {

    System.out.println(
            "No complaints available."
    );

} else {

    departmentCounts.forEach(
            (department, count) ->

                    System.out.println(
                            department
                            + " : "
                            + count
                    )
    );
}
```

```
// ================================================

// STATUS REPORT

// ================================================


System.out.println();


System.out.println(
        "---------------- STATUS REPORT ----------------"
);


if (statusCounts.isEmpty()) {


    System.out.println(
            "No status data available."
    );


} else {


    statusCounts.forEach(
            (status, count) ->


                    System.out.println(
                            status
                            + " : "
                            + count
                    )
    );
```

```
        }

        System.out.println();

        System.out.println(
                "Pending complaints : "
                + pending
        );

        System.out.println(
                "-----------------------------------------------"
        );
    }
}

// ============================================================
// MULTITHREADING
// ============================================================

public void simulateConcurrentSubmissions() {

    System.out.println();

    System.out.println(
            "=========== MULTITHREADING TEST ==========="
    );
```

```
// ============================================================
// CREATE DEMO CITIZEN
// ============================================================

synchronized (lock) {

    if (!citizens.containsKey(
            "C-DEMO")) {

        citizens.put(
                "C-DEMO",

                new Citizen(
                        "C-DEMO",
                        "Demo Citizen",
                        "9999999999",
                        "demo@example.com"
                )
        );
    }
}

// ============================================================
// INITIAL COMPLAINT COUNT
// ============================================================

final int initialComplaintCount;
```

```
synchronized (lock) {

    initialComplaintCount =
            complaints.size();
}

// ============================================================
// MONITOR THREAD
// ============================================================

Thread monitor =
        new Thread(
                () -> {

                    synchronized (lock) {

                        try {

                            while (
                                    complaints.size()
                                    == initialComplaintCount
                            ) {

                                lock.wait();
                            }
```

```
                    System.out.println(
                            "Monitor thread received "
                            + "notification: a new "
                            + "complaint was submitted."
                    );

                } catch (
                        InterruptedException e) {

                    Thread.currentThread()
                            .interrupt();

                    System.out.println(
                            "Monitor thread interrupted."
                    );
                }
            }
        },

        "Monitor-Thread"
    );

monitor.start();

// ============================================================
// THREAD TASK
// ============================================================
```

```
Runnable task =
        () -> {

            try {

                String threadName =
                        Thread.currentThread()
                                .getName();

                Complaint complaint =
                        submitComplaint(
                                "C-DEMO",
                                "ROADS",
                                "Concurrent road submission "
                                        + threadName,
                                Priority.MEDIUM
                        );

                System.out.println(
                        threadName
                        + " created complaint "
                        + complaint
                                .getComplaintId()
                );

            } catch (
```

```
                GrievanceException e) {

            System.out.println(
                    Thread.currentThread()
                            .getName()
                    + " : "
                    + e.getMessage()
            );
        }
    };

// ============================================================
// CREATE THREADS
// ============================================================

Thread t1 =
    new Thread(
            task,
            "Citizen-Thread-1"
    );

Thread t2 =
    new Thread(
            task,
            "Citizen-Thread-2"
    );
```

```
// ============================================================

// START THREADS

// ============================================================


t1.start();


t2.start();


// ============================================================

// WAIT FOR THREADS

// ============================================================


try {


    t1.join();


    t2.join();


    monitor.join();


} catch (

        InterruptedException e) {


    Thread.currentThread()

            .interrupt();


    System.out.println(
```

```
                "Concurrent simulation interrupted."
            );
        }

        System.out.println();

        System.out.println(
                "Thread-safe concurrent submission "
                + "test completed."
        );

        System.out.println(
                "================================================"
        );
    }
}

// ======================================================================
// GRIEVANCE EXCEPTION
// ======================================================================

class GrievanceException
        extends Exception {

    public GrievanceException(
            String message) {
```

```
        super(message);
    }
}

// ================================================================
// DUPLICATE COMPLAINT EXCEPTION
// ================================================================

class DuplicateComplaintException
        extends GrievanceException {

    public DuplicateComplaintException(
            String message) {

        super(message);
    }
}

// ================================================================
// DEPARTMENT UNAVAILABLE EXCEPTION
// ================================================================

class DepartmentUnavailableException
        extends GrievanceException {

    public DepartmentUnavailableException(
            String message) {
```

```
        super(message);
    }
}

// ============================================================
// INVALID INPUT EXCEPTION
// ============================================================

class InvalidInputException
        extends GrievanceException {

    public InvalidInputException(
            String message) {

        super(message);
    }
}

// ============================================================
// INVALID STATUS EXCEPTION
// ============================================================

class InvalidStatusException
        extends GrievanceException {

    public InvalidStatusException(
```

```
            String message) {

        super(message);
    }
}

// ======================================================================
// COMPLAINT NOT FOUND EXCEPTION
// ======================================================================

class ComplaintNotFoundException
        extends GrievanceException {

    public ComplaintNotFoundException(
            String message) {

        super(message);
    }
}

// ======================================================================
// END OF PROGRAM
// ======================================================================
```

7. Test Cases and Expected/Actual Results

TC ID	Test condition / input	Expected result	Actual result
TC01	Register citizen with valid name, phone, and email.	Citizen ID is generated and record is stored.	Passed: C1 generated and registration succeeded.
TC02	Submit complaint for citizen C1 to ROADS with HIGH priority.	Complaint is created with a unique ID and SUBMITTED status.	Passed: Complaint ID 1001 created.
TC03	Assign complaint 1001 to officer O101.	Assignment succeeds because O101 belongs to ROADS.	Passed: Officer assigned successfully.
TC04	Update complaint 1001 to IN_PROGRESS.	Status changes and history is appended.	Passed: Status updated successfully.
TC05	Track complaint 1001.	Complete complaint summary is displayed.	Passed: Status IN_PROGRESS and officer O101 displayed.
TC06	Generate reports after TC02–TC04.	Department, status, and pending counts are shown.	Passed: ROADS = 1; IN_PROGRESS = 1; pending = 1.
TC07	Run concurrent submission simulation.	Two threads receive unique IDs without map corruption; monitor is notified.	Passed: IDs 1002 and 1003 created; monitor notification received.
TC08	Use an unavailable department or missing citizen ID.	Relevant checked exception is displayed and program continues.	Implemented; should display DepartmentUnavailableException or InvalidInputException.
TC09	Move a complaint status backwards.	InvalidStatusException prevents the update.	Implemented; backward transition is rejected.

8. Execution Screenshots / Output

The following output was captured from an actual run after compiling the Java source. Students should add screenshots from their own IDE or terminal in the marked spaces below. Suggested screenshots include the main menu, citizen registration, complaint submission, officer assignment, report generation, and concurrent-thread execution.

8.1 Captured Console Output

```
================================================================
       GOVERNMENT PUBLIC GRIEVANCE MANAGEMENT SYSTEM
================================================================
1. Register citizen
2. Submit complaint
3. Assign officer
4. Update complaint status
5. Track complaint
6. Display all complaints
7. Generate reports
8. Simulate concurrent submissions
9. Exit
================================================================
Enter choice: 1
Navya
9876543210
navya@gmail.com

---------------- REGISTER CITIZEN ----------------
Citizen name: Phone number: Email: Generated Citizen ID: C1
Citizen registered successfully.
```

```
---------- TRACK COMPLAINT ----------
Complaint ID:
================ COMPLAINT DETAILS ================
Complaint ID : 1001
Citizen ID    : C1
Department    : ROADS
Description   : Road damage near college
Priority      : HIGH
Status        : IN_PROGRESS
Officer ID    : O101
Created At    : 2026-09-03T03:33:43.196764451
==================================================

---------- COMPLAINT HISTORY ----------
2026-09-03 03:33:43 - Complaint submitted with status SUBMITTED
2026-09-03 03:33:57 - Officer O101 assigned
2026-09-03 03:34:08 - Status changed to IN_PROGRESS
--------------------------------------------------
```

```
==========================================================
Enter choice: 7

---------- DEPARTMENT-WISE REPORT ----------
ROADS : 1

---------- STATUS REPORT ----------
IN_PROGRESS : 1

Pending complaints : 1
--------------------------------------------

==========================================================
       GOVERNMENT PUBLIC GRIEVANCE MANAGEMENT SYSTEM
```

```
==========================================================
Enter choice: 8

========== MULTITHREADING TEST ==========
Monitor thread received notification: a new complaint was submitted.
Citizen-Thread-2 created complaint 1003
Citizen-Thread-1 created complaint 1002

Thread-safe concurrent submission test completed.
==========================================

----------------------------------------------------------
```

.

```
========== MULTITHREADING TEST ===========
Monitor thread received notification: a new complaint was submitted.
Citizen-Thread-2 created complaint 1003
Citizen-Thread-1 created complaint 1002

Thread-safe concurrent submission test completed.
==========================================

============================================================
       GOVERNMENT PUBLIC GRIEVANCE MANAGEMENT SYSTEM
============================================================
1. Register citizen
2. Submit complaint
3. Assign officer
4. Update complaint status
5. Track complaint
6. Display all complaints
7. Generate reports
8. Simulate concurrent submissions
9. Exit
============================================================
Enter choice: 9

Thank you for using the Government Public Grievance Management System.
```

9. Analysis and Discussion

The design separates data entities from service operations, which improves readability and makes each responsibility explicit. Encapsulation is achieved through private fields and public methods that validate or control changes. Inheritance avoids repeating common identity fields, while method overriding allows each user type to report its own role.

HashMap supports direct lookup of citizens, officers, departments, and complaints by identifier. HashSet prevents duplicate officer identifiers in a department. ArrayList preserves complaint history in chronological order. Generics provide compile-time type safety, and Iterator is used when displaying complaint records. The report-generation logic traverses the complaint collection and aggregates counts with maps.

The concurrent submission demonstration uses two citizen threads and one monitor thread. A shared lock protects the collections and complaint ID generation. The monitor thread waits while no new complaint exists, and submitComplaint() calls notifyAll() after adding a complaint. This demonstrates mutual exclusion and inter-thread communication. The use of AtomicInteger further ensures unique complaint identifiers.

Limitations include in-memory storage, a console-only interface, basic email validation, and no authentication or database persistence. In a production system, records would be stored in a transactional database, users would be authenticated, role permissions would be enforced at the service boundary, and audit logs would be persisted.

The system contributes to SDG 16 by supporting transparent grievance handling and accountable public institutions. It contributes to SDG 9 through digital infrastructure, SDG 10 by giving citizens a structured channel to report service issues, and SDG 11 by supporting responsive urban civic services.

10. Conclusion

The Government Public Grievance Management System demonstrates how core Java concepts can be combined to solve a practical public-service problem. The application supports citizen registration, complaint submission, officer assignment, status tracking, reporting, concurrent processing, synchronization, inter-thread communication, and exception handling. The modular design is suitable for extension with a graphical interface, database persistence, authentication, notifications, and analytics.

11. Individual Contribution of Group Members

Group Member	Contribution
N.Navya Sri Vasavi(192411204)	Requirement analysis, problem statement, system design, documentation, OOP class design, encapsulation, inheritance, polymorphism, citizen registration and complaint management.
B.Laharisri(192421421)	Collections, generics, iterators, data management and reports, multithreading, synchronization, inter-thread communication, exception handling and testing.
All Members	Integration, debugging, screenshots, report finalization and presentation.

If the assignment is completed individually, replace the table with: “This project was completed individually. The student performed requirements analysis, design, implementation, testing, documentation, and presentation.”

SDG Mapping

The Government Public Grievance Management System using Java directly supports SDG 16 and also contributes to SDG 9, SDG 10, and SDG 11.

SDG Goal Relevance to the Project

SDG 16- Peace, Justice and Strong Institutions Improves transparency, accountability, citizen participation, and access to government grievance services.

SDG 9- Industry, Innovation and Infrastructure Uses Java and digital technology to create an efficient digital infrastructure for public grievance management.

SDG 10- Reduced Inequalities Provides citizens with a common platform to raise grievances and helps improve equal access to public services.

SDG 11- Sustainable Cities and Communities Supports better urban governance by enabling citizens to report problems related to public services and communities.

SDG 16 – Peace, Justice and Strong Institutions

12. References

[1] Oracle, “The Java Tutorials: Object-Oriented Programming Concepts,” https://docs.oracle.com/javase/tutorial/java/concepts/

[2] Oracle, “The Java Tutorials: Collections,” https://docs.oracle.com/javase/tutorial/collections/

[3] Oracle, “The Java Language Specification: Threads and Locks,” https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html

[4] United Nations, “Sustainable Development Goals,” https://sdgs.un.org/goals

[5] Course assignment brief, CSA0927 – Programming in Java, Government Public Grievance Management System.

13. One-Page Write-up

I contributed to the development of the Government Public Grievance Management System, a Java-based application developed to provide a structured and efficient mechanism for managing citizen grievances. My primary responsibilities included requirement analysis, system architecture, object-oriented class design, and implementation of core grievance management functionalities.

I analyzed the problem domain and identified the major functional requirements, including citizen registration, complaint submission, department management, officer assignment, complaint tracking, status management, and report generation. I designed the system using Java OOP principles, developing classes such as User, Citizen, Officer, Department, Complaint, and ManagementService. The implementation demonstrates abstraction, encapsulation, inheritance, polymorphism, and method overriding.

I was mainly involved in implementing the citizen registration and complaint submission processes. The system validates user inputs, verifies department availability, detects duplicate active complaints, generates unique complaint IDs using AtomicInteger, and maintains complaint status and history. I also utilized HashMap, HashSet, and ArrayList for efficient data organization and retrieval.

In addition, I contributed to exception handling, validation, testing, debugging, and system integration. Custom exceptions were incorporated to handle invalid inputs, duplicate complaints, unavailable departments, and missing records without terminating the application. I also assisted in preparing the algorithms, pseudocode, flowcharts, screenshots, and technical documentation.

Overall, my contribution focused on establishing the core architecture and reliable implementation of the grievance management system, while ensuring that the application followed object-oriented design principles and provided a clear, maintainable, and efficient solution.

Plagiarism Report: